



The .claude Folder Setup Guide

Turn Claude into a full dev team — automatically.

THE STRUCTURE

```
.claude/
■■■ agents/      ← Your AI team members
■■■ commands/    ← Custom slash commands
■■■ hooks/       ← Rules Claude MUST follow
■■■ rules/       ← Context-aware instructions
■■■ skills/      ← Situational intelligence
■■■ settings.json ← The control center

CLAUDE.md        ← The project brain (root)
```

1 AGENTS — Your AI Specialists

■ [.claude/agents/code-reviewer.md](#)

Each agent is a specialist. Claude delegates work to the right one automatically. Define tools, model size, memory, and max turns per agent.

● **code-reviewer** Reviews bugs & security before merge

● **debugger** Hunts down errors

● **test-writer** Writes regression tests

● **refactorer** Cleans up messy code

● **doc-writer** Generates documentation

● **security-auditor** Scans for vulnerabilities

Field	What it does
tools	Which tools the agent can access
model	sonnet, opus, or haiku
memory	Remembers context across sessions
maxTurns	Max steps before stopping

2 COMMANDS — Set Once, Run Anytime

■ `.claude/commands/fix-issue.md`

No retyping prompts. Define a command once and invoke it with a slash. Claude handles the full workflow end-to-end.

Example: Type `/fix-issue 42` → Claude reads the GitHub issue, finds relevant files, implements the fix, writes a regression test, runs `npm test`, and commits — all automatically.

● **/fix-issue** Fixes GitHub issues end-to-end

● **/deploy** Runs the full deployment pipeline

● **/pr-review** Reviews pull requests

3 HOOKS — Rules Claude Cannot Break

■ `.claude/hooks/pre-commit.sh`

Scripts that run before/after every action. Exit code 2 = blocked. Exit code 0 = allowed. These are hard guardrails.

● **pre-commit.sh** Blocks bad commits (type check → lint → test)

● **lint-on-save.sh** Auto-formats after every file edit

```
#!/bin/bash
npx tsc --noEmit          || exit 2
npx eslint <changed .ts files> || exit 2
npm test -- --silent       || exit 2
echo "All checks passed!"
```

4 RULES — Context-Aware Instructions

■ `.claude/rules/frontend.md`

Separate instruction files. Claude only loads rules relevant to the file you're currently editing — frontend rules for `.tsx` files, DB rules for queries, etc.

● **frontend.md** Component patterns, Tailwind, shadcn/ui

● **database.md** Query conventions, migrations

● **api.md** Auth, validation, error handling

5 SKILLS — Situational Intelligence

■ `.claude/skills/frontend-design/SKILL.md`

Instruction sets Claude applies when it recognises a specific situation. Also manually invocable. Great for encoding your team's exact design system, naming conventions, or architecture patterns.

Example: Ask Claude to build a landing page → it auto-applies your exact colour palette, spacing, and typography — no re-explaining needed.

6 CLAUDE.md — The Project Brain

■ `./CLAUDE.md` (project root)

Loaded every single session. This is your persistent memory. Put your stack, run commands, and code conventions here once and never repeat yourself again.

● **Stack** Next.js 14, Tailwind + shadcn/ui, Supabase, Stripe

● **Commands** npm run dev | build | test | lint

● **Conventions** TypeScript strict, Zustand for state, no `any`

● **Preferences** Dark mode first, auto-commit after every change

7 SETTINGS.JSON — The Control Center

■ `.claude/settings.json`

Controls what Claude is allowed and not allowed to do. Define allowed/denied bash commands, hook triggers, model selection, and memory settings.

■ ALLOW	■ DENY
<pre>npm run lint npm run test npm run build git diff gh issue</pre>	<pre>Read(.env) rm -rf git push --force</pre>

■ QUICK SETUP — Run This Once

```
mkdir -p .claude/{agents,commands,hooks,rules,skills/frontend-design}
touch .claude/agents/{code-reviewer,debugger,test-writer,refactorer,doc-writer,security-auditor}.md
touch .claude/commands/{fix-issue,deploy,pr-review}.md
touch .claude/hooks/{pre-commit,lint-on-save}.sh
touch .claude/rules/{frontend,database,api}.md
touch .claude/skills/frontend-design/SKILL.md
touch .claude/settings.json
touch CLAUDE.md
```

Source: resources.leadgenman.com/ClaudeFolderSetup

By **Manthan Patel** (@LeadGenMan)